

MetaSpend

System wizualizacji i analizy transakcji MetaMask Card

Artem Zakharov

Przedmiot: Systemowe Projektowanie

Grupa: A1

Rok akademicki: 2025/2026

1. Cel projektu

Celem projektu jest stworzenie funkcjonalnego oprogramowania umożliwiającego użytkownikom kart MetaMask Card śledzenie wydatków w sposób zbliżony do tradycyjnych aplikacji bankowych. Transakcje kryptowalutowe prezentowane w portfelu MetaMask są trudne do analizy pod kątem codziennych wydatków — brakuje kategoryzacji, podsumowań miesięcznych oraz przejrzystych wykresów.

CryptoTrack rozwiązuje ten problem poprzez:

- synchronizację historii transakcji z panelu MetaMask Card (rozszerzenie Chrome),
- przechowywanie i normalizację danych w centralnej bazie PostgreSQL,
- ręczną i automatyczną (AI) kategoryzację wydatków według kategorii i podkategorii,
- dashboard, listę transakcji oraz moduł analityki z wykresami i porównaniem okresów.

Ostatecznym efektem jest podniesienie świadomości finansowej użytkownika Web3 oraz ułatwienie kontroli budżetu opartego na wydatkach kartą.

2. Szczegółowy opis wymagań

2.1. Wymagania funkcjonalne

❖ Obsługa kont użytkowników

- Rejestracja i logowanie za pomocą adresu e-mail i hasła (JWT + sesje odświeżania).
- Opcjonalne logowanie portfelem Web3 (SIWE — Sign-In with Ethereum).
- Profil użytkownika: wyświetlana nazwa, powiązany adres portfela (PortfolioAccount).

❖ Synchronizacja transakcji MetaMask Card

- Rozszerzenie Chrome (Plasmo) pobiera listę transakcji ze stron `card.metamask.io` / `portfolio.metamask.io`.
- Parowanie rozszerzenia z kontem użytkownika kodem 6-cyfrowym wygenerowanym w ustawieniach aplikacji web.
- Endpoint REST `POST /api/v1/card-transactions/sync` — upsert transakcji po `externalId`.
- Zapisywane pola: `merchant`, `kwota fiat/crypto`, `status` (PENDING, SETTLED, DECLINED, REFUNDED), `znacznik czasu`.

❖ Zarządzanie transakcjami

- Lista transakcji z filtrowaniem po dacie, kategorii, merchantcie, statusie.
- Ręczna edycja kategorii, podkategorii i notatek dla pojedynczej transakcji.
- Statystyki zbiorcze: sumy wydatków, rozkład po kategoriach (`GET /transactions/stats`).

❖ Kategorie i podkategorie

- CRUD kategorii użytkownika (nazwa, kolor, ikona).
- Hierarchia parent → subcategory (np. „Jedzenie” → „Restauracje”).
- Kategorie systemowe (seed) oraz domyślne podkategorie.
- Strona „Categorize spendings” — masowa kategoryzacja merchantów.

❖ Automatyczna kategoryzacja (pamięć merchantów + reguły + AI)

- Model `cardMerchantMemory` — zapamiętanie mapowania `merchantKey` → kategoria po potwierdzeniu użytkownika lub AI.
- Reguły dopasowania merchantów (`firstMatchingCategoryId`, `packages/shared/src/merchant-rules/match.ts`) — proste dopasowanie „nazwa merchanta zawiera X”, pierwsza trafiona reguła wygrywa; uruchamiane po pamięci merchantów i przed AI.
- Usługa OpenAI przypisuje kategorię nieznanym merchantom (`bulk-categorize`).
- Śledzenie przebiegu w `CardCategorizationRun` (status, liczniki dopasowań).

❖ Analityka i wykresy

- Strona Analytics: wykresy wydatków w czasie, rozkład kategorii, porównanie okresów.
- Biblioteki: Recharts, agregacja po stronie API.
- Filtr zakresu dat (`DateRangePicker`).

❖ Dashboard

- Podsumowanie ostatnich transakcji, saldo karty (snapshot z rozszerzenia), szybki wgląd w wydatki.

❖ Podróże (Trips) — wielowalutowe zestawienia wydatków

- Grupowanie transakcji w podróże (model `Trip` ↔ `TripTransaction`, relacja N:N); nazwa, daty rozpoczęcia/zakończenia i domyślna waluta podróży.
- Waluta podróży wybierana automatycznie przy tworzeniu (najczęstsza, a

przy remisie — najwyższa sumą walut transakcji); użytkownik może ją później zmienić z listy (EUR/PLN/USD/GBP/CHF).

- Widok szczegółów podróży: suma w walucie natywnej każdej transakcji (`totalsByCurrency`) oraz suma i rozkład po kategoriach przeliczone na walutę podróży (`convertedTotal`, `categoryBreakdown`) wg kursów z `frankfurter.app` (cache ~24h, z degradacją do kursu 1:1 przy niedostępności API).

2.2. Wymagania нефunkcjonalne

❖ Łatwość użycia

- Responsywny interfejs (Next.js 15, React 19, Tailwind CSS, shadcn/ui).
- Sidebar nawigacyjny: Dashboard, Transactions, Categories, Settings; widok mobilny z menu.
- Tryb jasny/ciemny (ThemeSwitcher).

❖ niezawodność

- Monorepo (pnpm, Turbo): spójne typy w pakiecie `@crypto-tracker/shared`.
- Walidacja DTO (`class-validator`) i obsługa błędów HTTP w NestJS.
- Testy jednostkowe usług API (Jest).

❖ Wydajność

- PostgreSQL z indeksami na `userId`, `timestamp`, `categoryId`.
- Paginacja list transakcji; agregacje statystyk po stronie serwera.

❖ Bezpieczeństwo

- Hasła hashowane (bcrypt); tokeny JWT krótkotrwałe + refresh w bazie (hash).
- Tokeny rozszerzenia przechowywane jako hash (`ExtensionToken`).
- CORS i guardy autoryzacji na endpointach chronionych.
- TLS w środowisku produkcyjnym (HTTPS).

3. Ograniczenia

3.1. Ograniczenia technologiczne

- Wymagane połączenie sieciowe do synchronizacji z API oraz (przy kategoryzacji AI) do OpenAI.
- Rozszerzenie działa tylko w przeglądarce Chrome/Chromium z

uprawnieniami do domen MetaMask.

- Źródło danych ograniczone do transakcji MetaMask Card (nie pełna historia on-chain wszystkich sieci).

3.2. Ograniczenia zakresu

- Brak wieloużytkownikowych zespołów i ról menedżerskich (aplikacja single-user per konto).
- Brak integracji z systemami płatności fiat poza danymi z karty MetaMask.

3.3. Ograniczenia czasowe

- Projekt realizowany w ramach roku akademickiego 2025/2026.

4. Użytkownicy

4.1. Kluczowy użytkownik

❖ **Użytkownik karty MetaMask (właściciel konta)** — rejestruje się, paruje rozszerzenie, synchronizuje transakcje, zarządza kategoriami, przegląda analitykę i edytuje przypisania.

4.2. Użytkownicy techniczni

❖ **Administrator wdrożenia** — utrzymuje PostgreSQL, zmienne środowiskowe (DATABASE_URL, JWT_SECRET, OPENAI_API_KEY), deployment Docker.

5. Nazewnictwo i definicje

- Użytkownik** — osoba posiadająca konto w systemie CryptoTrack.
- Transakcja karty** — operacja zarejestrowana na koncie MetaMask Card (merchant, kwota, status).
- Merchant** — nazwa punktu handlowego (znormalizowany klucz merchantKey).
- Kategoria / podkategoria** — etykieta klasyfikacji wydatku (hierarchia drzewiasta).
- Pamięć merchantów (CardMerchantMemory)** — trwałe mapowanie merchant → kategoria.
- Rozszerzenie Chrome** — moduł Plasmo scrapujący HTML listy transakcji MetaMask.
- SIWE** — Sign-In with Ethereum; logowanie podpisem portfela.
- JWT** — JSON Web Token używany do autoryzacji żądań API.

6. Diagram przypadków użycia (UML)

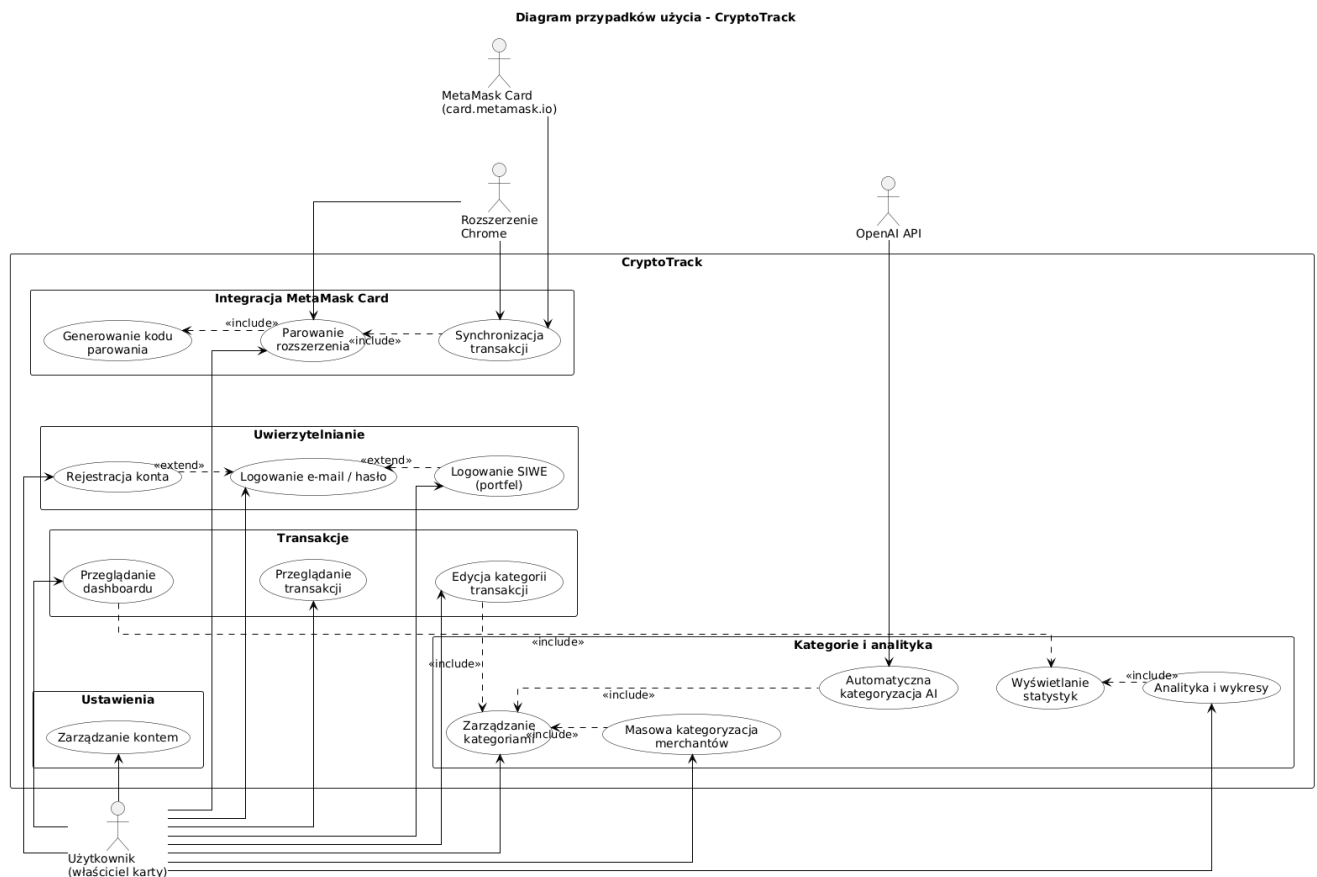


Diagram obejmuje aktorów: Użytkownik, Rozszerzenie Chrome oraz przypadki: rejestracja, SIWE, synchronizacja, przeglądanie transakcji, kategorie, kategoryzacja AI, analityka, parowanie rozszerzenia.

7. Scenariusze przypadków użycia

7.1. Rejestracja nowego użytkownika

Pole	Wartość
Nazwa przypadku użycia	Rejestracja nowego użytkownika
Cel	Utworzenie konta w systemie CryptoTrack
Warunki wstępne	Użytkownik nie jest zalogowany; e-mail nie jest zajęty
Warunek pomyślnego	Konto utworzone, możliwe

zakończenia	logowanie
Stan końcowy — niepowodzenie	Komunikat błędu (np. zajęty e-mail, słabe hasło)
Główni aktorzy	Użytkownik
Wywołanie	Wejście na /auth/login → „Zarejestruj się”

Główny przepływ:

1. Użytkownik otwiera formularz rejestracji.
2. Wprowadza e-mail, hasło, opcjonalnie nazwę wyświetlaną.
3. System waliduje dane i zapisuje użytkownika w tabeli `users`.
4. Użytkownik zostaje przekierowany do logowania lub automatycznie otrzymuje tokeny.

7.2. Parowanie rozszerzenia Chrome

Pole	Wartość
Nazwa przypadku użycia	Parowanie rozszerzenia
Cel	Powiązanie rozszerzenia z kontem użytkownika
Warunki wstępne	Użytkownik zalogowany w aplikacji web
Warunek pomyślnego zakończenia	Token rozszerzenia zapisany w <code>chrome.storage</code>

Główny przepływ:

1. W Settings użytkownik generuje 6-cyfrowy kod parowania.
2. W popup rozszerzenia wpisuje kod i klika Connect.
3. API weryfikuje kod (`POST /auth/extension/pair`) i zwraca bearer token.

7.3. Synchronizacja transakcji

Pole	Wartość
Nazwa	Synchronizacja transakcji karty

przypadku
użycia

Warunki
wstępne

Rozszerzenie sparowane; otwarta karta
MetaMask w przeglądarce

Warunek
pomyślnego
zakończenia

Transakcje widoczne w /transactions

1. Użytkownik otwiera card.metamask.io i klika „Sync now” w rozszerzeniu.
2. Content script scrapuje wiersze tabeli transakcji.
3. Background normalizuje dane i wysyła POST sync z tokenem.
4. API wykonuje upsert w tabeli `transactions`.

7.4. Automatyczna kategoryzacja wydatków

1. Użytkownik wchodzi w Categories → Categorize spendings.
2. System skanuje nieprzypisane merchanty; najpierw dopasowuje z CardMerchantMemory.
3. Niedopasowane merchanty przechodzą przez silnik reguł (`firstMatchingCategoryId`); dopasowane od razu otrzymują kategorię.
4. Pozostałe (nieprzypisane przez reguły) merchanty kierowane do OpenAI; wynik zapisywany i uczy pamięci.
5. Użytkownik widzi postęp w CardCategorizationRun.

7.5. Przeglądanie analityki

1. Użytkownik otwiera /analytics.
2. Wybiera zakres dat.
3. API zwraca agregaty; frontend renderuje wykresy (Recharts).

Diagram czynności - rejestracja użytkownika

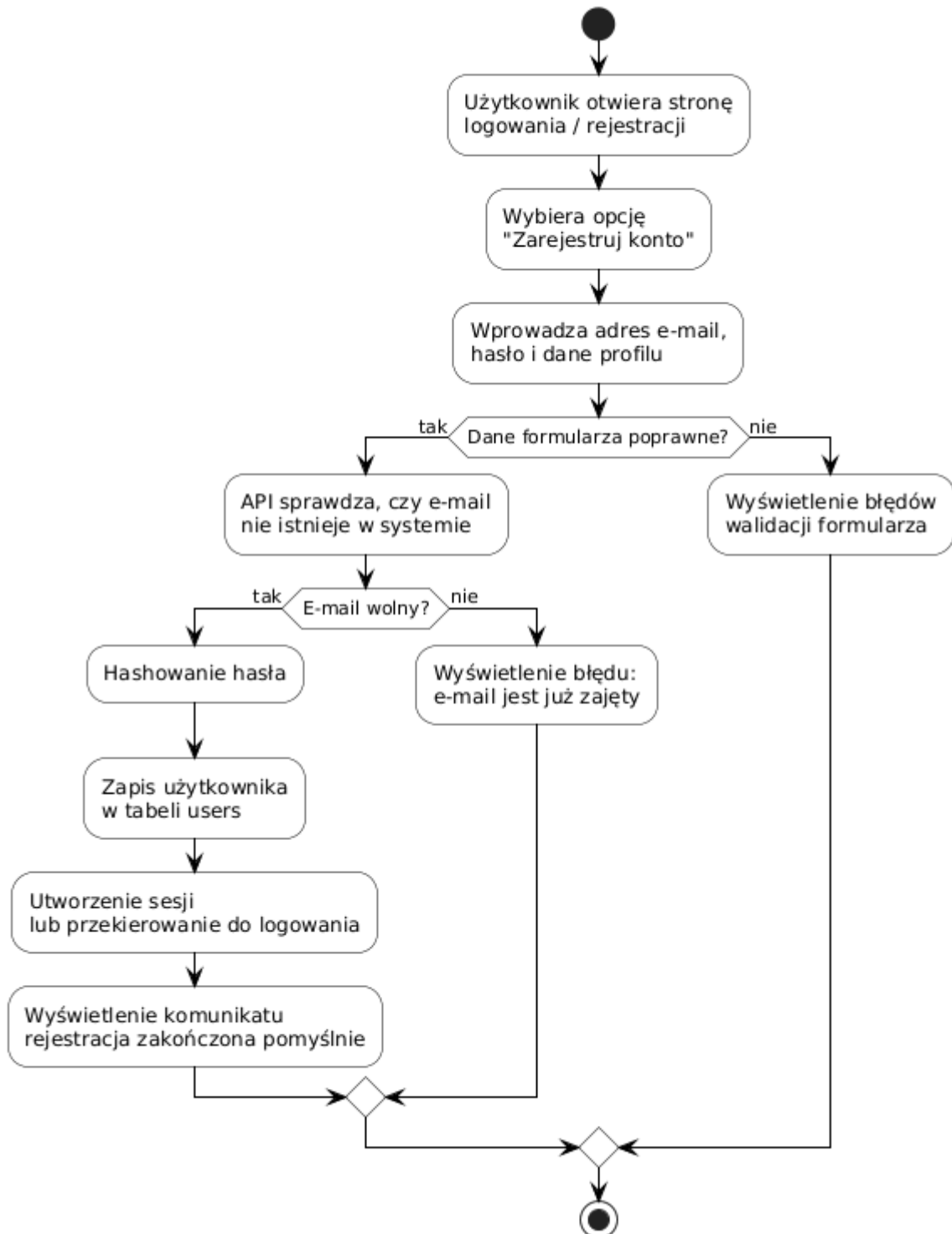


Diagram czynności: Rejestracja użytkownika

Diagram czynności - parowanie rozszerzenia Chrome

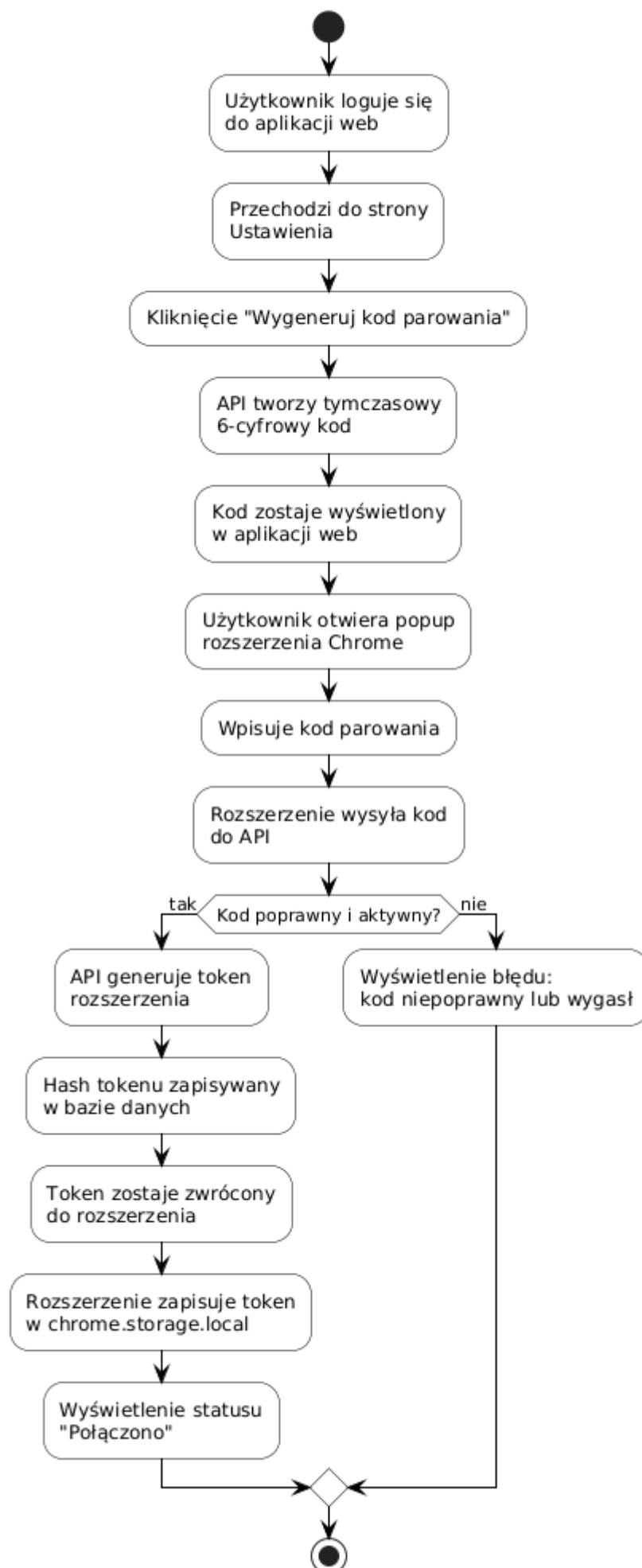


Diagram czynności: Parowanie rozszerzenia

Diagram czynności - synchronizacja transakcji

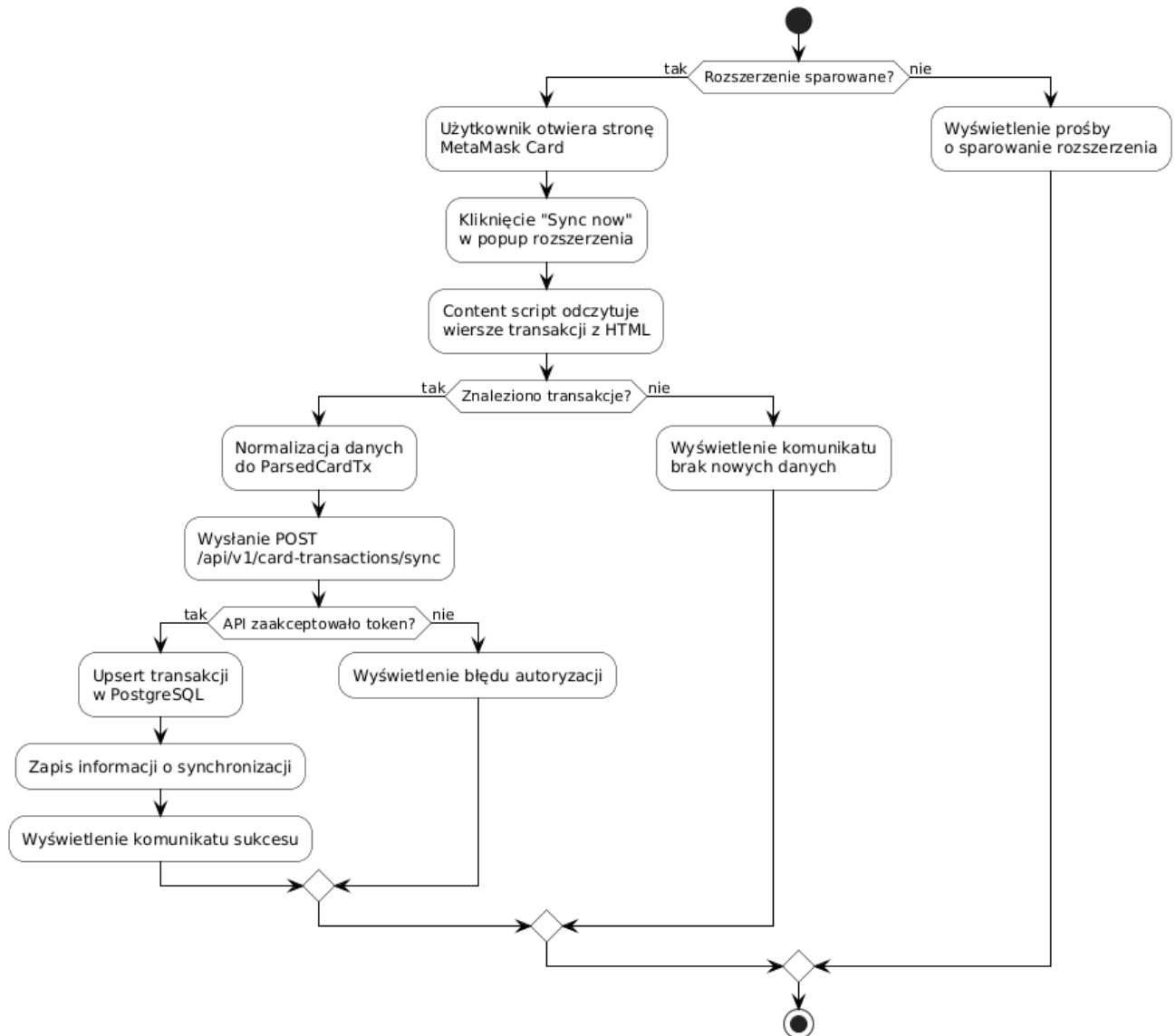


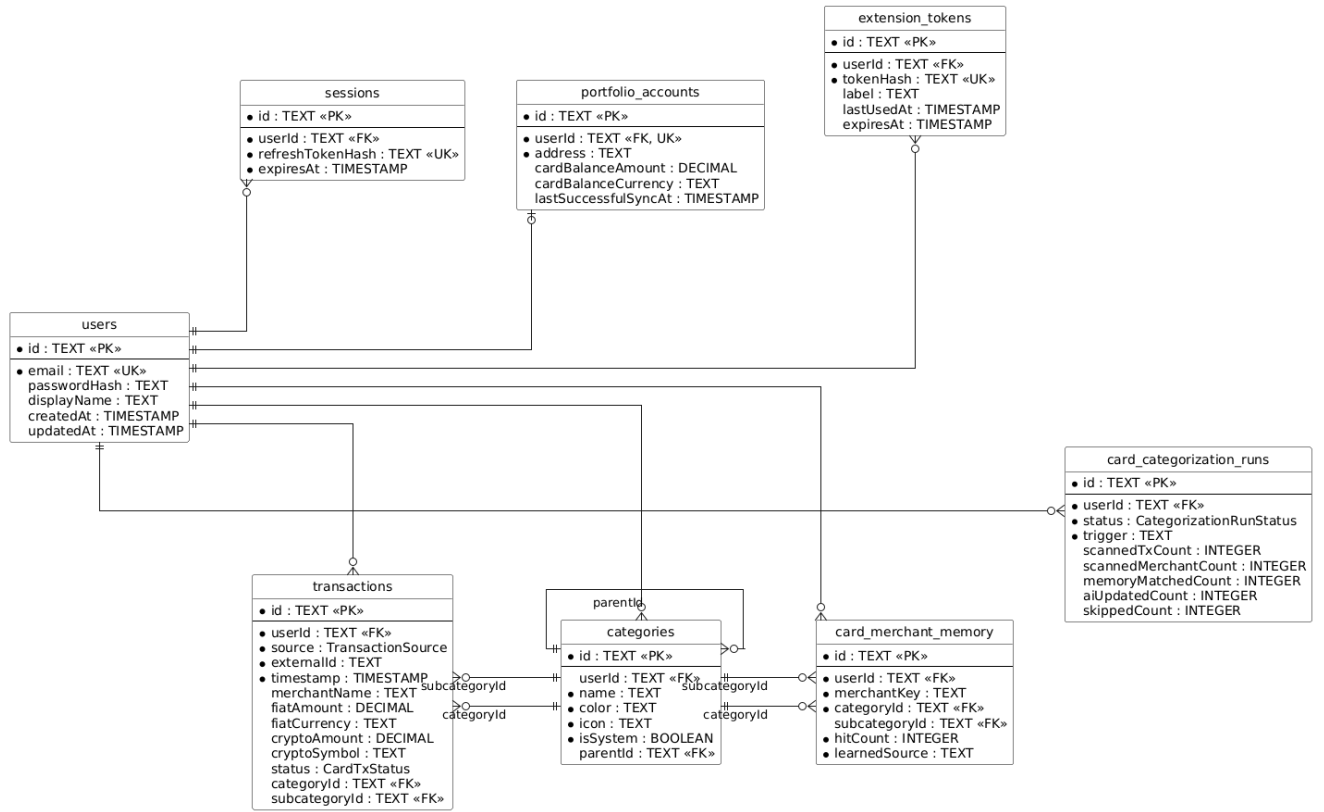
Diagram czynności: Synchronizacja transakcji

Diagram czynności - automatyczna kategoryzacja AI

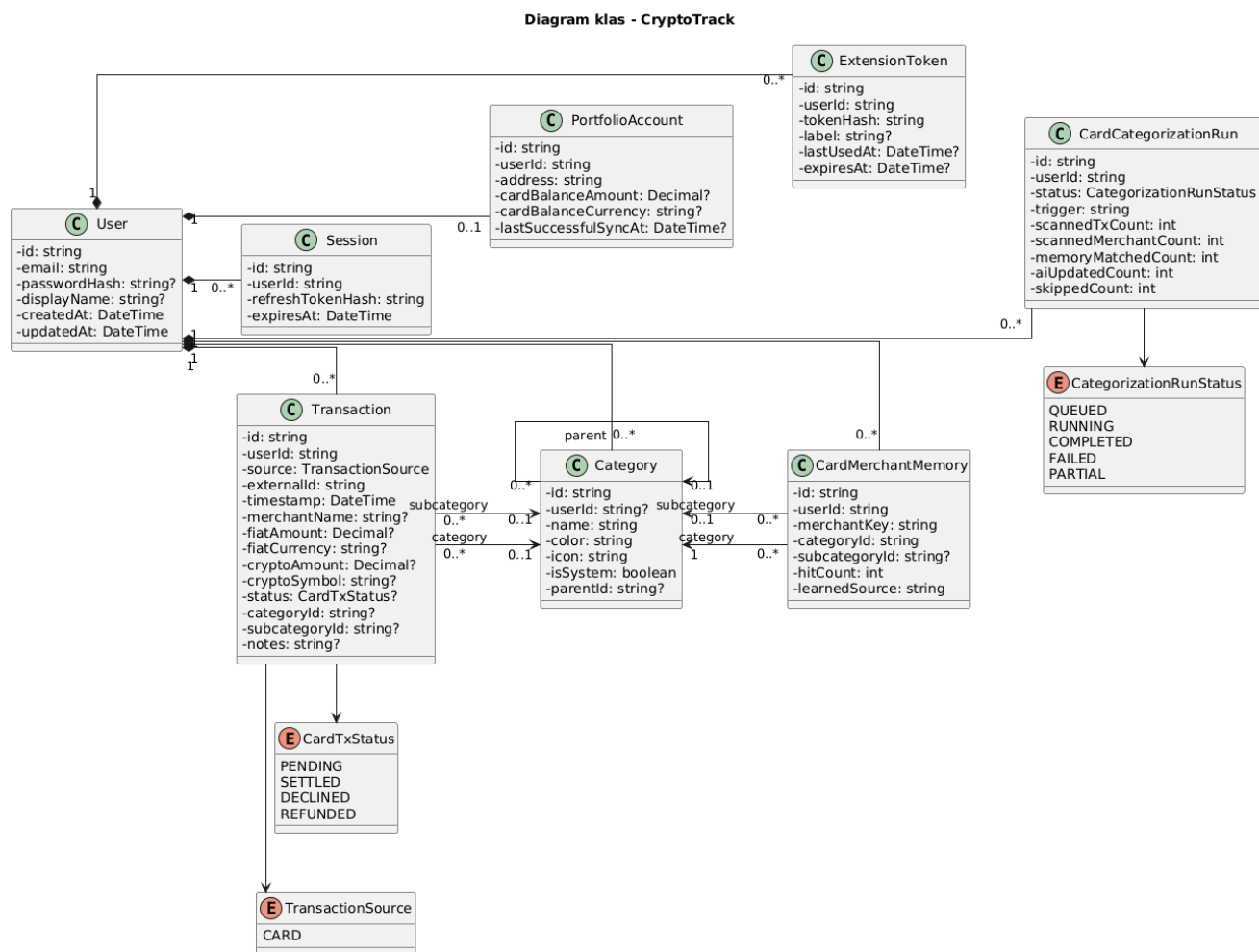


9. Diagram ERD

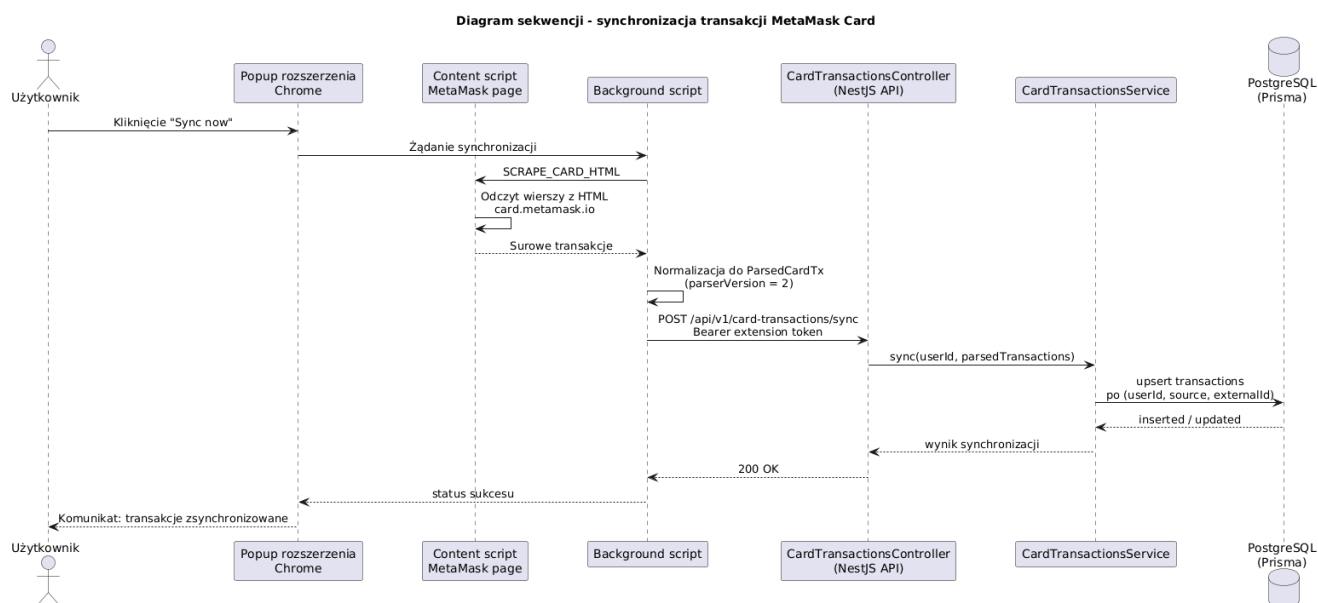
Diagram ERD - CryptoTrack



10. Diagram klas (UML)

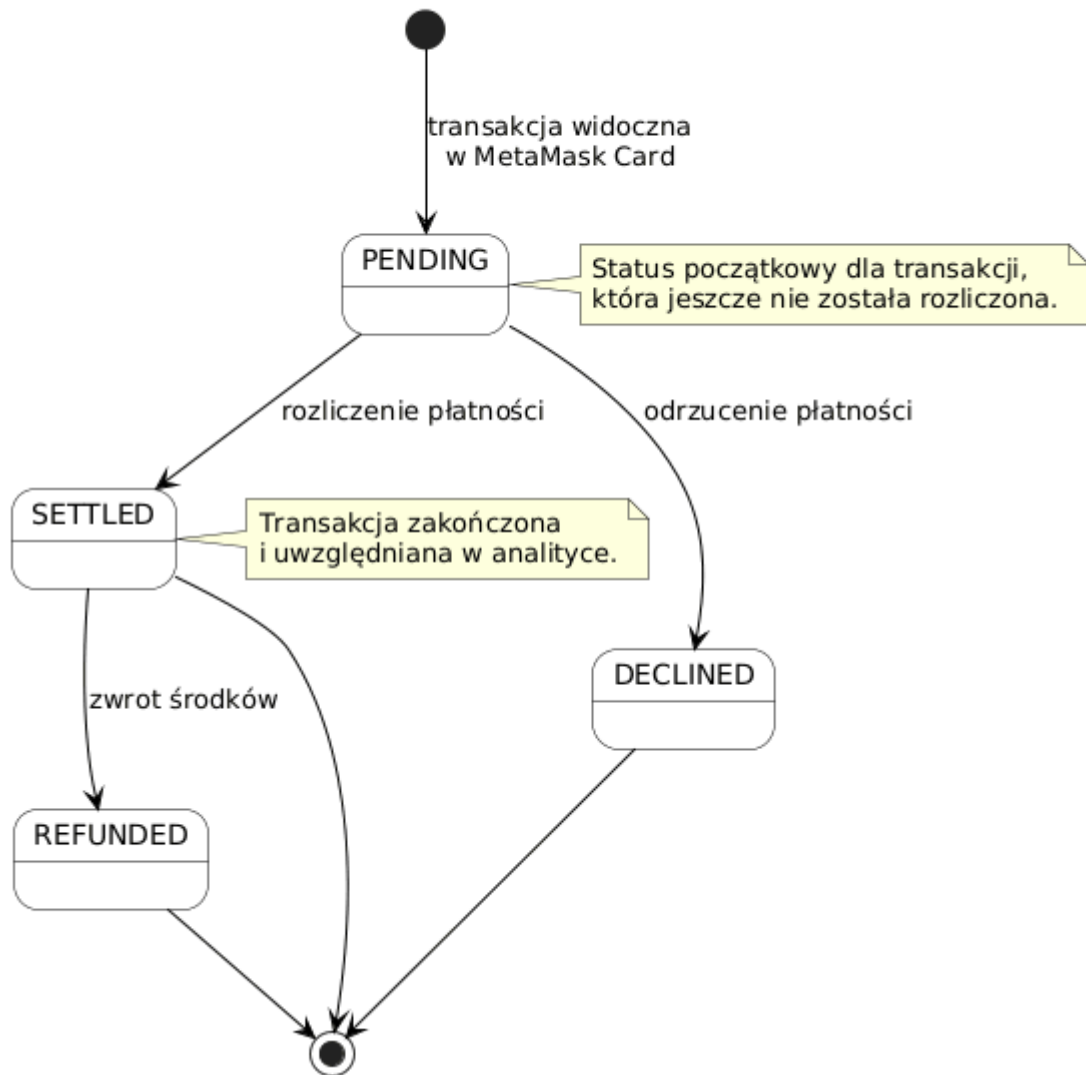


11. Diagram sekwencji (UML)

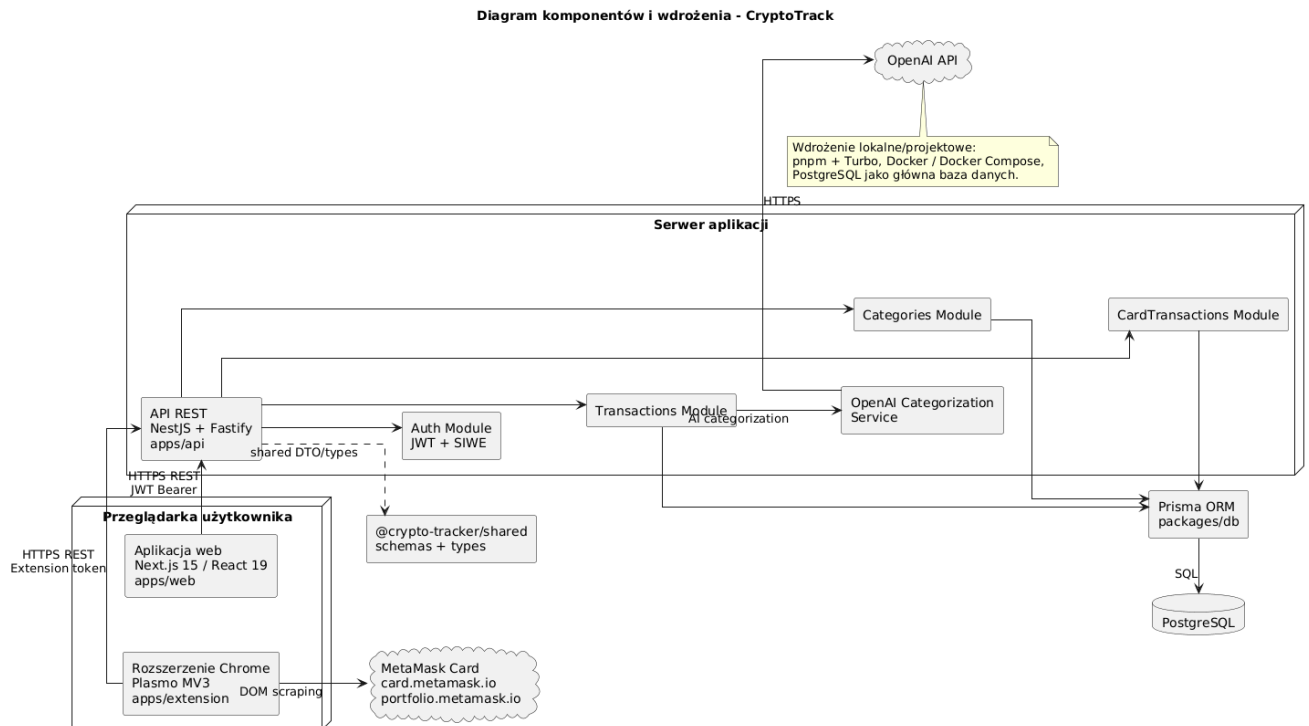


12. Diagram stanów (UML)

Diagram stanów - transakcja MetaMask Card



13. Diagram komponentów i wdrożenia (UML)



14. Plan procesu testowania

14.1. Testy automatyczne

- **Testy jednostkowe** — Jest: `card-transactions.service.spec.ts`, `card-merchant-auto-categorize.spec.ts`.
- **Testy integracyjne** — (planowane) endpointy auth i sync z testową bazą.

14.2. Testy manualne

Scenariusze opisane w rozdziale 19 — wykonywane przez testera na środowisku dev (web + API + rozszerzenie).

14.3. Utrzymanie

Po wdrożeniu: raportowanie błędów, poprawki w kolejnych wersjach, migracje Prisma.

15. Raport z analizy rynku

15.1. Funkcje kluczowe CryptoTrack

- Specjalizacja w MetaMask Card (nie ogólny eksplorator blockchain).
- Kategoryzacja wydatków z AI i pamięcią merchantów.
- Rozszerzenie do synchronizacji bez ręcznego importu CSV.

15.2. Porównanie z konkurencją

❖ Koinly / CoinTracker

- Zalety: szerokie wsparcie podatków i wielu giełd.
- Wady: mniej nastawione na codzienne wydatki kartą DeFi; wyższy próg wejścia.

❖ Rotki

- Zalety: open source, lokalna kontrola danych.
- Wady: wymaga ręcznej konfiguracji; brak dedykowanego flow MetaMask Card.

❖ Aplikacje bankowe tradycyjne

- Zalety: gotowa kategoryzacja i UX.
- Wady: nie obejmują wydatków kartą krypto MetaMask.

15.3. Podsumowanie

CryptoTrack wypełnia niszę „expense tracker dla MetaMask Card”. Wybór narzędzia zależy od tego, czy użytkownik potrzebuje raportów podatkowych on-chain, czy przejrzystego budżetu kartowej.

16. Struktura bazy danych

16.2. Model logiczny

Encje: User (1) — (N) Transaction, Category, Session, ExtensionToken, CardMerchantMemory, CardCategorizationRun, Trip; User (1) — (1) PortfolioAccount; Category (1) — (N) SubCategory (self-reference parentId); Trip (N) — (N) Transaction przez TripTransaction.

16.3. Model fizyczny

PostgreSQL; ORM Prisma; tabele: users, sessions, transactions, categories, card_merchant_memory, card_categorization_runs, extension_tokens, portfolio_accounts.

17. Dokumentacja bezpieczeństwa

17.1. Szyfrowanie w tranzycie

HTTPS (TLS 1.2+) między przeglądarką a API w produkcji.

17.2. Szyfrowanie w spoczynku

Hasła: bcrypt hash. Tokeny rozszerzenia i refresh: przechowywane jako hash w bazie. Szyfrowanie dysku — zależne od dostawcy hostingu PostgreSQL.

17.3. Uwierzytelnianie

JWT access + refresh token w tabeli sessions; opcjonalnie SIWE dla portfela Ethereum.

17.4. Ochrona przed atakami

- Walidacja wejścia (DTO, class-validator) — ochrona przed injection.
- Prisma parameterized queries — ochrona przed SQL injection.
- Rate limiting (planowane na produkcji).

17.5. Uprawnienia

Model single-tenant: każdy użytkownik widzi wyłącznie swoje transakcje (`userId` w każdym zapytaniu).

18. Kod SQL (PostgreSQL)

Poniżej fragment schematu wygenerowanego przez Prisma Migrate (dialekt PostgreSQL).

```
CREATE TABLE "users" (  
  "id" TEXT NOT NULL,  
  "email" TEXT NOT NULL,  
  "passwordHash" TEXT,  
  "displayName" TEXT,  
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  "updatedAt" TIMESTAMP(3) NOT NULL,  
  CONSTRAINT "users_pkey" PRIMARY KEY ("id")  
);  
  
CREATE UNIQUE INDEX "users_email_key" ON "users"("email");  
  
CREATE TABLE "transactions" (  
  "id" TEXT NOT NULL,  
  "userId" TEXT NOT NULL,  
  "source" "TransactionSource" NOT NULL DEFAULT 'CARD',  
  "externalId" TEXT NOT NULL,  
  "timestamp" TIMESTAMP(3) NOT NULL,  
  "merchantName" TEXT,  
  "fiatAmount" DECIMAL(24,8),  
  "categoryId" TEXT,  
  "subcategoryId" TEXT,  
  CONSTRAINT "transactions_pkey" PRIMARY KEY ("id"),
```

```

        CONSTRAINT "transactions_userId_fkey" FOREIGN KEY ("userId")
REFERENCES "users"("id") ON DELETE CASCADE
);
CREATE UNIQUE INDEX "transactions_userId_source_externalId_key"
    ON "transactions"("userId", "source", "externalId");

```

Pełny schemat: `packages/db/prisma/schema.prisma` oraz katalog `prisma/migrations/`.

18.2. Widok przykładowy (PostgreSQL)

```

CREATE VIEW v_user_spending_by_category AS
SELECT
    t."userId",
    c.name AS category_name,
    DATE_TRUNC('month', t."timestamp") AS month,
    SUM(t."fiatAmount") AS total_fiat
FROM transactions t
LEFT JOIN categories c ON c.id = t."categoryId"
WHERE t."fiatAmount" IS NOT NULL
GROUP BY t."userId", c.name, DATE_TRUNC('month', t."timestamp");

```

19. Przypadki testowe

19.1. Rejestracja — niespójne hasło

Nr	Krok	Rezultat
1	Otwórz /auth/login → rejestracja	Formularz rejestracji
2	Wpisz różne hasła w polach Hasło i Powtórz	—
3	Kliknij Zarejestruj	Komunikat błędu walidacji; brak konta

19.2. Rejestracja — pomyślna

Nr	Krok	Rezultat
1	Poprawne dane, — unikalny e-mail	—
2		
3	Zarejestruj	Przekierowanie / możliwość logowania

19.3. Synchronizacja bez parowania

Nr	Krok	Rezultat
1	Sync now bez tokenu	Komunikat błędu w popup rozszerzenia

20. Testy jednostkowe

```
► CardTransactionsService
  ✓ upserts card rows into transactions and deduplicates repeated external ids in one sync (1.903347ms)
  ✓ updates an existing card transaction by external id on later syncs (0.397946ms)
✓ CardTransactionsService (2.75256ms)
► CategoriesService
  ✓ returns only user-owned categories and creates fresh defaults with subcategories idempotently (0.860686ms)
  ✓ adds default subcategories only to matching existing parent categories (0.232732ms)
  ✓ rejects duplicate names only among sibling categories (0.429607ms)
✓ CategoriesService (1.644305ms)
► TransactionsService (card)
  ✓ reports positive card spend in monthly stats when PLN is single fiat (1.100231ms)
  ✓ filters transactions and stats by parent categories and subcategories independently (0.280503ms)
  ✓ groups card merchants and updates all matching rows + merchant memory (0.49975ms)
✓ TransactionsService (card) (2.018746ms)
► cardMerchantAutoCategorize schema helpers
  ✓ stripJsonFence removes markdown fences (0.512123ms)
  ✓ parseAutoCategorizeAiPayload parses valid payloads (1.155105ms)
  ✓ reject extra top-level keys (strict) (0.526471ms)
  ✓ validateAssignmentsForChunk enforces full coverage (0.206102ms)
✓ cardMerchantAutoCategorize schema helpers (2.948894ms)
► CardCategorizationRunService AI assignment
  ✓ writes category + merchant memory when AI assigns non-null ids (0.732221ms)
  ✓ returns zeros when nothing to categorize (0.208306ms)
✓ CardCategorizationRunService AI assignment (1.042901ms)
i tests 14
i suites 5
i pass 14
i fail 0
i cancelled 0
i skipped 0
i todo 0
i duration_ms 1395.815241
```

Testy obejmują m.in. normalizację merchantów i logikę auto-kategoryzacji (`card-merchant-auto-categorize.spec.ts`).

21. Podręcznik użytkownika

Spis treści: 1. Rejestracja. 2. Logowanie. 3. Parowanie rozszerzenia. 4. Synchronizacja. 5. Transakcje. 6. Kategorie. 7. Kategoryzacja AI. 8. Analityka. 9. Ustawienia.

21.1. Rejestracja i logowanie

 **MetaSpend**

Sign in

Create account

Email

you@example.com

Password

.....

Confirm password

.....

Create account

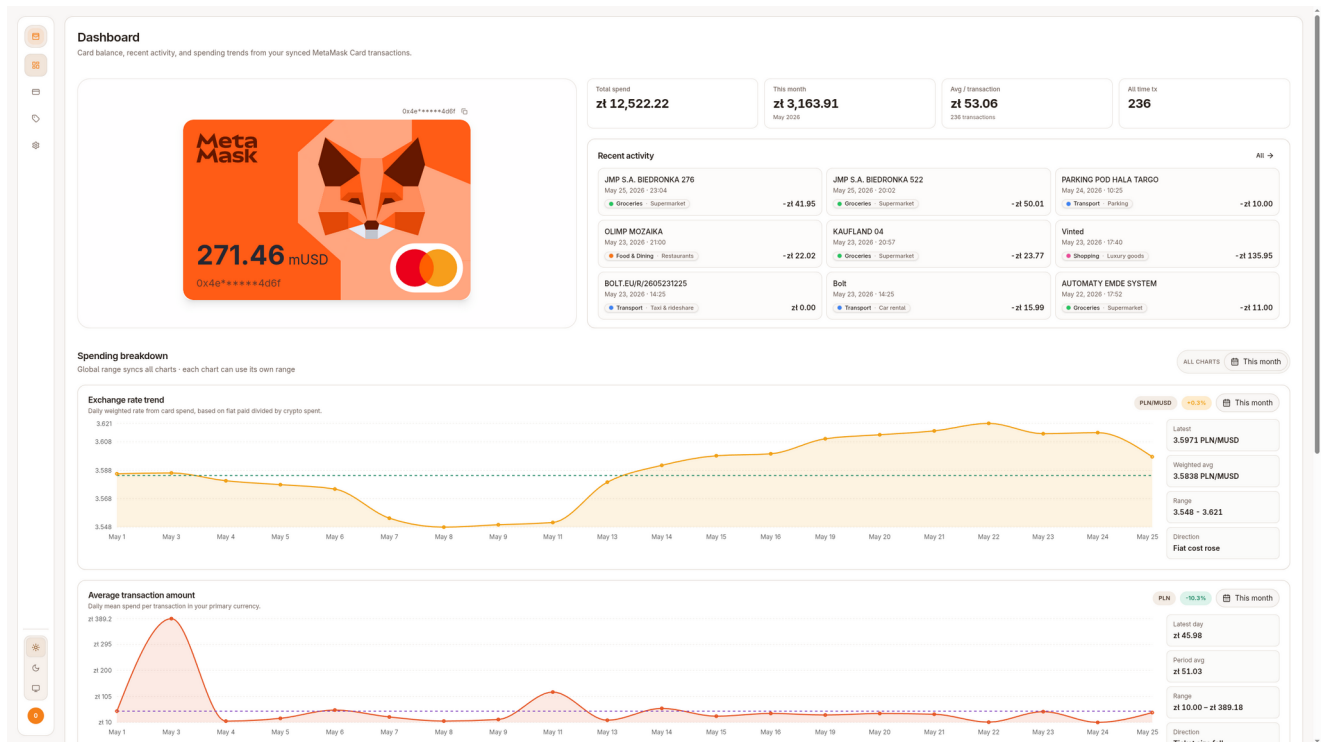
or continue with

● 0x4e55...4d6f

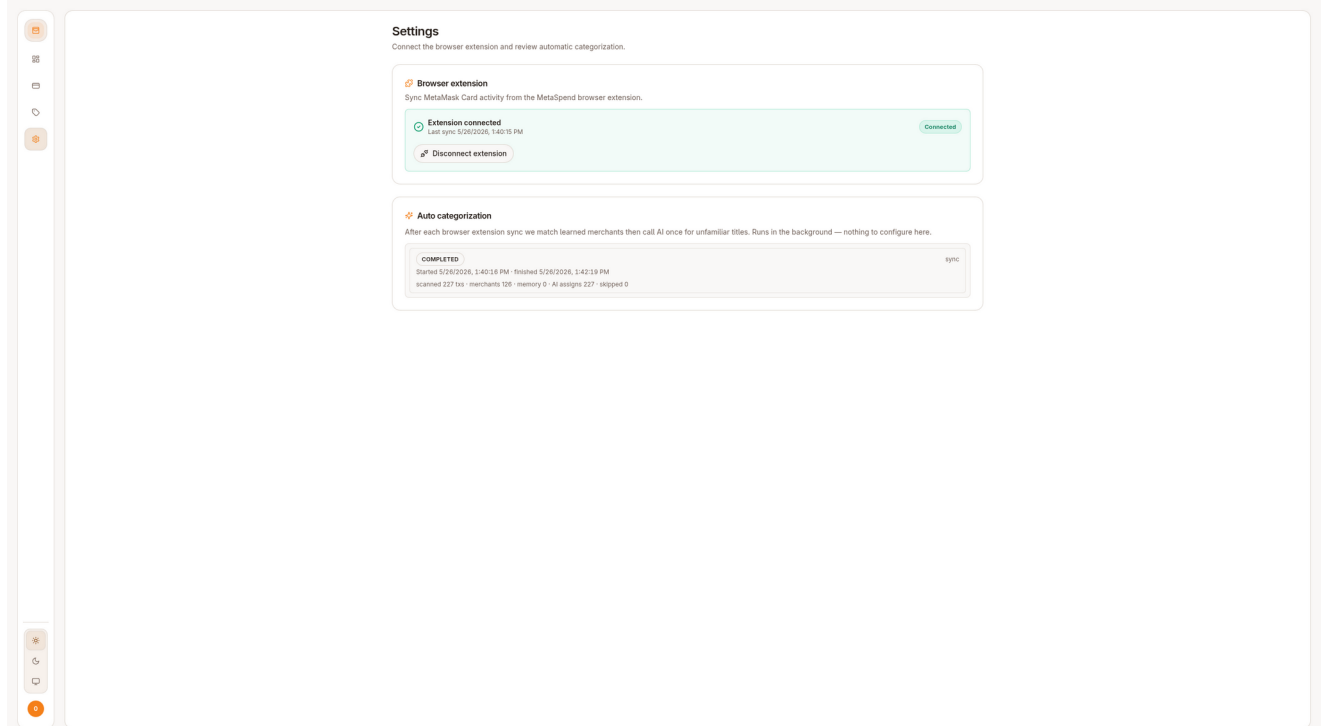
Disconnect

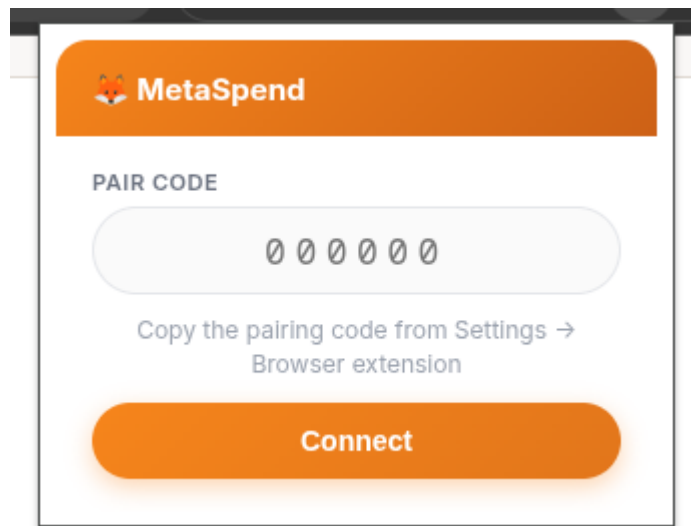
 Sign in with Ethereum

21.2. Dashboard



21.3. Parowanie rozszerzenia





21.4. Synchronizacja transakcji

	Search merchant or notes...	All statuses	All categories	All time
	Total transactions: 236 Average exchange rate: 3.6093 mixed pairs Average transaction amount: zł 53.06			
Monday, May 25				-zł 91.96
JMP S.A. BIEDRONKA 276	Groceries	Supermarket	SETTLED	-zł 41.95
SETTLED				outflow
11:04:00 PM	-11.6764 MUUSD	gas 0.003354 MUUSD	3.5927 PLN/MUUSD	
JMP S.A. BIEDRONKA 522	Groceries	Supermarket	SETTLED	-zł 50.01
SETTLED				outflow
8:02:00 PM	-13.6889 MUUSD	gas 0.003386 MUUSD	3.6007 PLN/MUUSD	
Sunday, May 24				-zł 10.00
PARKING POD HALA TARGO	Transport	Parking	SETTLED	-zł 10.00
SETTLED				outflow
10:25:00 AM	-2.767 MUUSD	gas 0.003371 MUUSD	3.614 PLN/MUUSD	
Saturday, May 23				-zł 197.73
OLIMP MOZAIKA	Food & Drink	Restaurants	SETTLED	-zł 22.02
SETTLED				outflow
9:00:00 PM	-6.0926 MUUSD	gas 0.003294 MUUSD	3.6142 PLN/MUUSD	
KAUFLAND 04	Groceries	Supermarket	SETTLED	-zł 23.77
SETTLED				outflow
8:57:00 PM	-6.5945 MUUSD	gas 0.003296 MUUSD	3.61 PLN/MUUSD	
Vinted	Shopping	Luxury goods	SETTLED	-zł 135.95
SETTLED				outflow
9:40:00 PM	-37.6168 MUUSD	gas 0.003286 MUUSD	3.6141 PLN/MUUSD	
BOLT EUR/2605231225	Transport	Taxi & rideshare	PENDING	zł 0.00
PENDING				outflow
2:25:00 PM	0 PLN			
Bolt	Transport	Car rental	SETTLED	-zł 15.99
SETTLED				outflow
2:25:00 PM	-4.4287 MUUSD	gas 0.003222 MUUSD	3.6197 PLN/MUUSD	
Friday, May 22				-zł 11.00
AUTOMATY EMDE SYSTEM	Groceries	Supermarket	SETTLED	-zł 11.00
SETTLED				outflow
9:52:00 PM	-3.0392 MUUSD	gas 0.003387 MUUSD	3.6208 PLN/MUUSD	

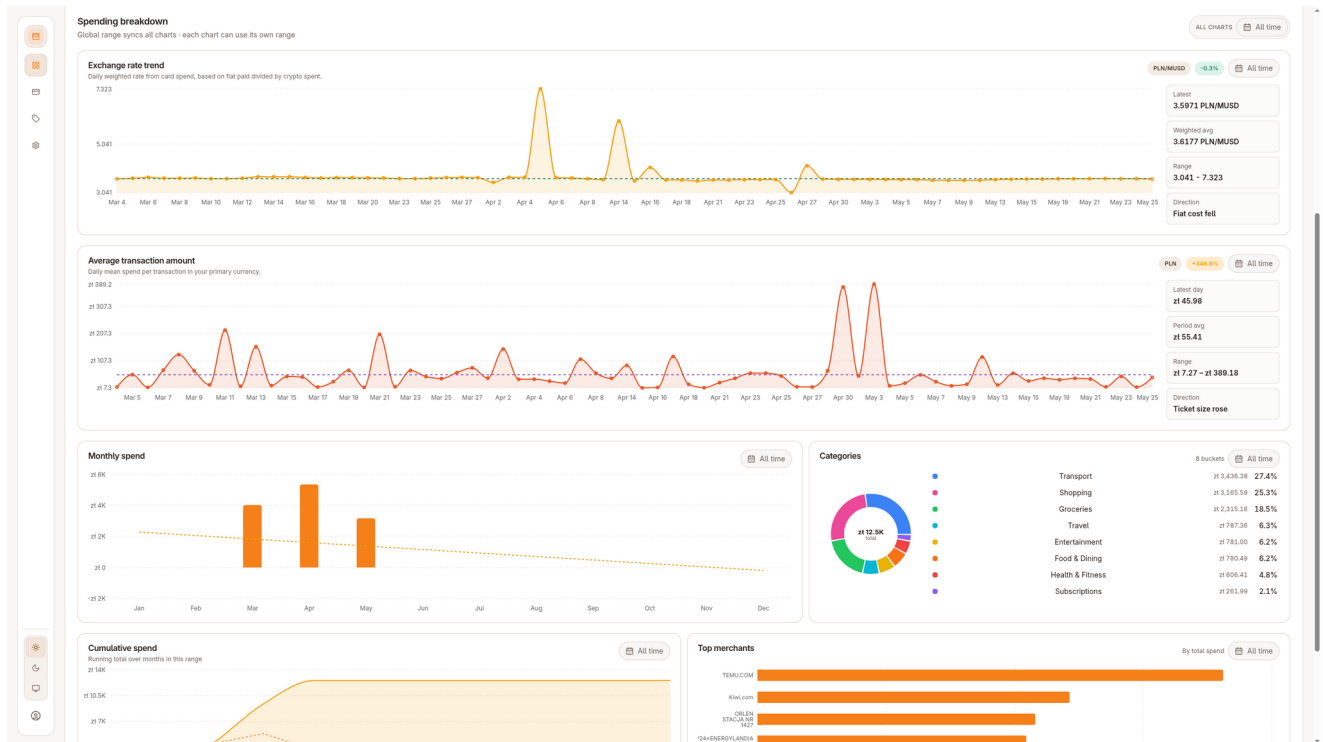
21.5. Lista transakcji

Q Search merchant or notes...		All statuses	All categories	All time
Total transactions: 236		Average exchange rate: 3.6093 mixed pairs		Average transaction amount: zł 53.06
Monday, May 25 -zł 91.96				
JMP S.A. BIEDRONKA 276 • Groceries • Supermarket • SETTLED				
SETTLED	11:04:00 PM	-11.6764 MU\$D	gas 0.003384 MU\$D	3.5927 PLN/MU\$D -zł 41.95 outflow
JMP S.A. BIEDRONKA 522 • Groceries • Supermarket • SETTLED				
SETTLED	8:02:00 PM	-13.8589 MU\$D	gas 0.003386 MU\$D	3.6007 PLN/MU\$D -zł 50.01 outflow
Sunday, May 24 -zł 10.00				
PARKING POD HALA TARGO • Transport • Parking • SETTLED				
SETTLED	10:25:00 AM	-2.767 MU\$D	gas 0.003371 MU\$D	3.614 PLN/MU\$D -zł 10.00 outflow
Saturday, May 23 -zł 187.73				
OLIMP MOZAIKA • Food & Dining • Restaurants • SETTLED				
SETTLED	9:00:00 PM	-6.0926 MU\$D	gas 0.003294 MU\$D	3.6142 PLN/MU\$D -zł 22.02 outflow
KAUFLAND 04 • Groceries • Supermarket • SETTLED				
SETTLED	8:57:00 PM	-6.5845 MU\$D	gas 0.003286 MU\$D	3.6141 PLN/MU\$D -zł 23.77 outflow
Vented • Shopping • Luxury goods • SETTLED				
SETTLED	9:40:00 PM	-37.8165 MU\$D	gas 0.003288 MU\$D	3.6141 PLN/MU\$D -zł 135.95 outflow
BOLT.EUR/2605231225 • Transport • Taxi & ride share • PENDING				
PENDING	2:25:00 PM	0 PLN		zł 0.00 outflow
Bolt • Transport • Car rental • SETTLED				
SETTLED	2:25:00 PM	-4.4287 MU\$D	gas 0.003222 MU\$D	3.6097 PLN/MU\$D -zł 15.99 outflow
Friday, May 22 -zł 11.00				
AUTOMATY EMDE SYSTEM • Groceries • Supermarket • SETTLED				
SETTLED	9:52:00 PM	-5.0392 MU\$D	gas 0.003367 MU\$D	3.6236 PLN/MU\$D -zł 11.00 outflow

21.6. Zarządzanie kategoriami

My categories 68 total		+ New category
Create labels for card spend. Transactions auto-categorize in the background.		
Bills & Utilities 6 + Sub		
Electricity		
Gas		
Internet		
Phone & mobile		
TV & cable		
Water		
Coffee & Drinks 4 + Sub		
Cafe		
Coffee chain		
Juice & smoothies		
Tee house		
Entertainment 6 + Sub		
Amusement parks		
Cinema & theater		
Concerts & events		
Museums & galleries		
Nightlife		
Sports events		
Food & Dining 4 + Sub		
Bakery		
Fast food		
Food delivery		
Restaurants		
Groceries 5 + Sub		
Convenience store		
Market & bazaar		
Organic & health food		
Supermarket		

21.7. Analityka



Stos technologiczny (podsumowanie)

TypeScript, React 19, Next.js 15, Tailwind CSS, NestJS, Fastify, PostgreSQL, Prisma, JWT, SIWE, REST API, rozszerzenie Chrome (Plasmo), OpenAI API, Docker, pnpm, Turbo.

Repozytorium

GitHub: <https://github.com/ZAKHAROV-Artem/meta-spend>